大模型训练横纵分析报告

研究对象：大规模分布式训练（并行策略、显存优化、训练框架、计算加速）

2026 年 5 月更新：NVIDIA Blackwell 平台 NVFP4 训练正式投入生产——在不损失精度前提下实现 3 倍训练加速和近 2 倍的训练性价比提升。Megatron Core 发布 MoE 可扩展训练技术报告，MoE Parallel Folding 实现异构并行映射。DeepSpeed v0.18.9 进一步优化 ZeRO 与 SuperOffload。"Towards Fully FP8 GEMM LLM Training at Scale" 将全 FP8 训练从理论推向实践。合成数据驱动的预训练范式持续改变 Scaling 逻辑。

类型：AI 工程 / 基础设施

方法论：横纵分析法（Horizontal-Vertical Analysis）

撰写时间：2026-04-30

目录

1. 一句话定义
2. 技术背景
3. 纵向分析：大规模训练的进化史

- 单卡与 DP 时代（2018-2019）→ 并行化的序章（2020-2021）→ ZeRO 革命（2020-2021）→ 3D Parallelism 成为标准（2021-2022）→ FSDP 与 PyTorch 的统一（2023-2024）→ DeepSeek 的效率奇迹（2024-2025）→ IO 觉醒：FlashAttention → 硬件基础设施的演进

1. 横向分析：2026 年的训练格局

- 四大训练框架选型 · 五个核心框架深度分析 · 并行策略选择图谱 · 通信瓶颈深度分析 · 训练成本急剧下降 · MoE vs Dense 全面对比 · 训练云平台

1. 横纵交汇洞察

- 训练工程的核心矛盾 · 效率提升的四个来源 · 历史如何塑造竞争格局 · 被抛弃的路线 · 未来训练会变成什么 · 一条贯穿始终的暗线

1. 实战：面试题、场景题、应用题

- 5 道面试题 · 3 道场景题 · 1 道应用题

1. 信息来源

一、一句话定义

大规模分布式训练，就是让模型在合理的时间内学会它该学会的东西。而训练框架与基础设施，则是支撑这一过程的"多卡协作操作系统"——它帮用户管理调度（什么卡算什么）、通信（卡之间怎么传数据）、存储（参数和中间结果放哪里），让用户只需写模型代码。

训练只做一次（或者少数几次），但这一次的成本可能高达几千万美元。GPT-3 175B 在 2020 年训练一次估计花了 1200 万美元，LLaMA 3 405B 在 2024 年花了超过 5000 万美元。训练看重的是吞吐——GPU 利用率推满、通信开销压低、训练时间最短。推理要的是响应速度，训练要的是吞吐量——这两个优化方向完全不同，所用的并行策略和技术栈也大相径庭。

核心矛盾：模型越来越大（从 340M 的 BERT 到 671B 的 DeepSeek-V3，五年 2000 倍增长），但单 GPU 的显存容量增长远跟不上模型参数的增长。H100 80GB 能放下一个 BERT 模型几十次，但连一个 70B 模型的 FP16 参数（140GB）都装不下。所有分布式训练技术——数据并行、张量并行、流水线并行、ZeRO 优化、3D 并行——归根到底都是在跟这个矛盾周旋。

另一个更微妙的矛盾：训练效率（MFU, Model FLOPs Utilization）在过去两三年里从 ~35% 提升到了 ~70%，但绝对训练成本并没有等比下降——因为大家把省下来的算力拿去训更大的模型了。效率提升的直接受益者是模型质量，不是钱包。

> 🎯 **读完这篇你能**：理解数据并行、张量并行、流水线并行和 ZeRO 各自解决什么问题，能在给定模型规模和 GPU 数量的情况下判断该用哪种并行策略组合。

二、技术背景

> 完整的 Transformer 架构前置知识请参阅 Transformer 前置知识。以下是与大规模训练直接相关的核心概念速查。

数据并行（Data Parallelism, DP）：最简单也最直观的并行方式。每个 GPU 持有完整的模型副本，各自处理不同的数据 batch。前向传播各自独立，反向传播后通过 all-reduce 通信聚合梯度，所有 GPU 的模型参数保持一致。通信开销 = O(model_size)，因为每步都要同步完整梯度。

张量并行（Tensor Parallelism, TP）：把单个 Transformer 层的参数沿某个维度切开，分布到多张 GPU 上。例如把注意力层的 QKV 投影矩阵按列切开，每张 GPU 算一部分。每张 GPU 计算结束后需要通过 all-reduce 通信合并结果。TP 的通信量极大（每层 forward 两次 all-reduce），因此要求 GPU 之间高速互联——NVLink（~900 GB/s）是最低要求。TP 通常不跨节点。

流水线并行（Pipeline Parallelism, PP）：把模型的不同层放在不同 GPU 上，输入数据像流水线一样依次经过各层。通信量小（只需传输中间激活），但存在流水线气泡（pipeline bubble）——最早的 micro-batch 进入流水线时，后几层的 GPU 在空等。气泡率约 10-20%，PP 深度越大气泡比例越高。

ZeRO（Zero Redundancy Optimizer）：微软 DeepSpeed 的核心贡献。核心洞察——训练时显存最大消耗者不是模型参数，而是 optimizer states（Adam 的 momentum + variance 各占模型大小的 1 倍，总共 2x 模型大小）。ZeRO-1 只切分 optimizer states，ZeRO-2 再加切分梯度，ZeRO-3 加切分模型参数。ZeRO-3 的理论显存节省可达 16x+，但代价是每一步都需要额外的通信来收集参数。

混合精度训练（Mixed Precision Training）：模型参数和激活用 FP16 存储（节省显存），梯度计算和优化器更新用 FP32（保持精度）。2018 年由 Micikevicius 等人系统化，现已成为所有大模型训练的标配。DeepSeek-V3 更进一步，在 2024 年首次在千卡规模实现了 FP8 混合精度训练。

MFU（Model FLOPs Utilization）：衡量训练系统效率的黄金指标。定义为实际达到的 FLOPs 除以硬件理论峰值 FLOPs。MFU 越高说明 GPU 计算资源利用越充分。2020 年 GPT-3 的 MFU 估计在 30-40%，2024 年 DeepSeek-V3 达到了 ~70%。

这些概念是理解后续所有分析的基础。大规模训练的演进史，就是围绕这些技术瓶颈的突围史。

2.1 解决方案的三层结构

整个训练系统的技术栈可以拆解为三个层面：

并行策略层面：数据并行（DP）→ 参数分片（ZeRO/FSDP）→ 张量并行（TP）→ 流水线并行（PP）→ 序列并行（SP）→ 专家并行（EP）。每一层解决一个不同的"放不下"问题：模型太大（TP/PP）、显存太浪费（ZeRO）、序列太长（SP）、Expert 太多（EP）。

计算优化层面：FlashAttention 1→2→3 通过 IO-aware 算法将 Attention 的显存带宽需求从 $O(N^2)$ 降到 $O(N)$。混合精度训练从 FP32 到 FP16/BF16 到 FP8，精度每降一级，吞吐量几乎翻倍。

硬件架构层面：NVLink 提供节点内 600-900GB/s 的带宽，而跨节点的 InfiniBand 只有 50GB/s——差了 10 倍以上。这决定了通信密集的策略（TP）必须放在节点内部，通信量小的策略（PP、ZeRO）才能跨节点。训练框架的核心工作，就是在这些硬件约束下做最优的资源分配。

📚 **关联报告**

- Transformer — 训练的对象是 Transformer 架构，理解架构是理解训练的前提
- 大模型推理部署 — 训练与推理共享底层优化技术，但优化目标截然不同
- 大模型数据工程 — 数据是训练的原料，Chinchilla 法则连接了两者
- AI 芯片格局 — 万卡集群的算力硬件基础

三、纵向分析：大规模训练的进化史

3.1 单卡与 DP 时代（2018–2019）：模型小，一切都很简单

2018 年的训练很简单。BERT Base 110M 参数，一张 V100 32GB 就能训，一次完整训练在 4–16 个 Cloud TPU 上跑 4 天。BERT Large 340M 需要稍多资源，但 4 张 V100 也够了。那个年代做训练的人最关心的是收敛速度和精度调参——硬件根本不是问题。

2019 年 2 月，OpenAI 发布了 GPT-2 1.5B。这在当时是"巨型模型"，但仍然可以用数据并行在单机多卡上训练。OpenAI 用了 32 个 TPUv3 core，做了两周。训练本身稳定——主要挑战不在分布式，而在数据处理和采样策略。

同年，NVIDIA 发布了 **APEX（A PyTorch Extension）**，把混合精度训练（FP16 + FP32）带入了主流。核心思想很简单：大部分计算用 FP16（速度和显存优势），关键位置（loss scaling、梯度更新）用 FP32 兜底。这篇工作发表时没有引起轰动——因为它只是把 best practice 封装成了工具。但回头看，APEX 的发布标志着一个转折：**大家开始意识到 FP32 是浪费的**。当时没有人预见，六年之后 FP8 训练会在千卡集群上稳定运行。

这个时期还有一个关键的认知转变：**Data Parallel 就够了**。2019 年最大的模型（GPT-2 1.5B）在 FP16 下只占约 3GB，单张 V100（32GB）远没有装满。大家的口号是"买更多的 GPU，用 DP 堆更多的吞吐"。没有人想到，一年后这个假设会被 GPT-3 175B 彻底打碎。

3.2 并行化的序章（2020–2021）：模型太大，一张卡装不下了

2020 年 6 月，OpenAI 发表了 GPT-3 论文。175B 参数、96 层、d_model=12288、FFN 中间维度 49152。关键数字：FP32 下 700GB。即使 FP16，也至少 350GB——12 张 V100 32GB 都不够放。数据并行在这里失效了：你买再多 GPU，单卡放不下模型参数，一切无从谈起。

整个行业被迫重新思考一个问题：**模型参数本身装不下了，怎么办?**

2020 年 9 月，NVIDIA 发布了 **Megatron-LM**。这篇论文（"Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM"）给出了一个简单而实用的答案：**张量并行**。

Megatron-LM 的核心是把 Transformer 层的参数切开。以 MLP 层为例：正常情况下是 `GeMM(X, A) -> GeLU -> GeMM(H, B)`。Megatron 把 A 沿列切成两份：`A = [A1, A2]`，两个 GPU 分别计算 `X @ A1` 和 `X @ A2`，然后做 all-reduce 拼接结果。注意力层也类似：把 QKV 投影矩阵按列切开。关键是 Megatron 设计了一个计算-通信重叠的调度：前向传播时两个 GPU 可以并行计算各自的切片，只需要在 ffn 的中间和末尾做两次 all-reduce。实际上，在 NVIDIA 的测试中，Megatron-LM 在 8 张 GPU 上实现了近乎线性的加速比——对于那个年代的行业来说，这是一个突破性的结果。

Megatron-LM 训练了 NVIDIA 的第一个 8.3B 模型（在当时已经算很大了）。训练在 512 张 V100 上跑了数周，用 TP=8 + DP=64 的组合。这实际上是 3D 并行的早期雏形——虽然它当时只包含了 TP 和 DP 两个维度。

同一时期，Google 的 **GShard**（2020 年 6 月）和 GSPMD（2021 年 3 月）为训练提供了自动并行化的编译器方案。GShard 可以在编译阶段自动决定如何切分计算图到不同的设备。它的思想后来被 JAX 的 `pmap` 和 `pjit` 继承。但 GShard 最大的贡献不是它的编译技术，而是它在训练中引入了**条件计算（Conditional Computation）**——这实际上是 MoE 训练的最早系统化实现。

2020–2021 年，并行化的概念从学术界走到了工业界。但两条路线——NVIDIA 的手工并行化（Megatron）和 Google 的自动并行化（GShard/GSPMD）——选择了完全不同的哲学。NVIDIA 说"工程师手写并行策略，性能最优"，Google

说"编译器自动决定，工程成本最低"。这种分歧延续至今，没有真正的胜者。

3.3 ZeRO 革命（2020-2021）：微软的显存解药

2020 年 2 月，微软发布了 DeepSpeed 库，附带一篇论文——**"ZeRO: Memory Optimizations Toward Training Trillion Parameter Models"**。这篇论文后来成为分布式训练领域引用量最高的论文之一。原因很简单：它发现了一个所有人都忽略了的问题。

训练时显存消耗有三块：模型参数、梯度、optimizer states。对于 Adam optimizer 来说，optimizer states 的显存消耗是 2x 模型大小（momentum 和 variance 各占一份）。加上梯度（1x）和参数（1x），FP16 训练的总显存消耗是 4x 模型大小 + 中间变量。一个 100B 模型在 FP16 下光参数/梯度/optimizer states 就要占 400GB。当时大家的注意力全在"怎么切模型"上，没人想过"这些 optimizer states 是不是冗余的"。

ZeRO 的核心洞察：**optimizer states 在 DP 中是完全冗余的**。数据并行下每张 GPU 都维护一份完整的 optimizer states（Adam 的 momentum + variance），但 optimizer 更新只需要一份。去掉了冗余，显存需求直接砍半。

DeepSpeed ZeRO 有三个级别：

- **ZeRO-1**：只切分 optimizer states。每张 GPU 只维护一部分 optimizer states，通信量极小（只在参数更新前做一次 gather）。显存节省约 4x。
- **ZeRO-2**：切分 optimizer states + 梯度。梯度通信改为 reduce-scatter（相比于 all-reduce，reduce-scatter 的输出是分散在各卡上的，不需要每卡都持有完整梯度）。显存节省约 8x。
- **ZeRO-3**：切分全部三项：参数、梯度、optimizer states。每张 GPU 只持有模型的一部分参数，需要时通过 all-gather 收集。显存节省超 16x，但通信量大幅增加——每步 forward/backward 都需要多次 all-gather/scatter。

ZeRO 的思路跟 TP/PP 完全不同：它不是从"计算怎么分摊"的角度思考，而是从"存储怎么去冗余"的角度。它是系统优化，不是算法优化。ZeRO 不需要改模型结构，不需要改训练代码，只需要在 DP 通信策略上做文章。这种"低侵入性"让它很快被社区接受。

2021 年 8 月，微软用 DeepSpeed ZeRO 训练了 **Turing-NLG 530B**——人类历史上第一个万亿参数级别的语言模型。530B 参数在 FP16 下约 1TB，ZeRO-3 让它在 512 张 A100（80GB）上完成训练。当时所有媒体都在报道"万亿参数"，但真正重要的不是 530B 这个数字，而是 ZeRO 证明了**你不需要特殊的硬件或框架就能训万亿参数模型**。

DeepSpeed 的社区影响力从此建立。它在 PyTorch 上运行，不需要改模型结构，加上几行配置就能用——这和 Megatron-LM 烦琐的手工切分形成了鲜明对比。从此，DeepSpeed 和 Megatron 的绑定成了行业标准：**Data Parallel 维度用 DeepSpeed（ZeRO），Tensor/Pipeline Parallel 维度用 Megatron**。这个组合一直延续到今天。

3.4 3D Parallelism 成为标准（2021-2022）

2021-2022 年，"3D Parallelism"成为大规模训练的标配。3D 指的是三个并行的维度：

- **数据并行（DP）**：跨节点通信（通过 DeepSpeed ZeRO 或标准 DP），使用廉价以太网（~100 Gbps）
- **张量并行（TP）**：节点内通信（通过 Megatron-LM），必须使用 NVLink（~900 GB/s）
- **流水线并行（PP）**：节点内或跨节点（通过 Megatron-LM），通信量小，但气泡问题突出

三个维度叠加，训练系统变得非常复杂：每个 GPU 在每一步中要参与多个 all-reduce、reduce-scatter、all-gather、点对点通信，通信模式因并行策略组合而异。

2022 年 4 月，Google 发布了 **PaLM 540B**。这是当时最大的稠密（Dense）Transformer 模型。训练用了 6144 块 TPUv4 芯片，分布在 768 个 Pod 上。并行策略：TP=8（每个 Pod 内）+ PP=16（跨 Pod）+ DP=4096。训练耗时约 2 个月。

PaLM 的训练是教科书式的 3D 并行案例。TP=8 在一个 TPUv4 Pod 内（芯片间高速互联），PP=16 将模型分为 16 个 stage，跨 Pod 传输——每个 stage 内的芯片组通过 DP 同步。实际训练中遇到了大量问题：芯片故障率高于预期（6144 块芯片平均每天坏几块），需要频繁 checkpoint + 自动恢复；流水线气泡在 PP=16 时达到 ~15%，等效于 900 多块芯片在空转。

PaLM 训练的经验直接影响了后续模型的设计。Google 在 PaLM 之后的模型（PaLM 2、Gemini）上不再使用这么大的 PP 深度，转而通过增加 TP 或使用更高效的芯片互联来减少跨 stage 通信。

这个时期行业开始认识到一个关键问题：**3D 并行的超参数优化（怎么分 TP、PP、DP）对训练效率的影响巨大，但最优配置跟模型结构、硬件拓扑、通信带宽密切相关**。没有一个通用的配置公式。每个模型在每次新集群上训练前，都需要做一次"并行策略搜参"。

与此同时，MoE 训练也在这一阶段成熟。2022 年 1 月，Google 发布了 **Mixture of Experts (MoE) 训练** 的系统优化工作——实际上是 GShard 的延续。MoE 训练增加了一个新的并行维度：**专家并行（Expert Parallelism, EP）**。EP 的问题是 all-to-all 通信——不同专家分布在不同的 GPU 上，token 需要在 GPU 之间跳转。通信模式从 DP 的 all-reduce（一对多）变成了 all-to-all（多对多），网络压力完全不同。

MoE 训练的另一个核心问题是**负载均衡**——让 token 均匀分布到各个 expert 上，不要出现某个 expert 处理了 90% 的 token 而其他 expert 闲置的情况。Google 的做法是在 MoE 路由层加入 auxiliary loss（辅助损失），惩罚不均匀路由。这个辅助损失的权重是训练的超参数——权重太大影响模型质量，太小负载不均衡。这个矛盾的解决，是后来 DeepSeek 的重要贡献之一。

3.5 FSDP 与 PyTorch 的统一（2023-2024）

2023 年 2 月，Meta 在 PyTorch 2.0 中发布了 **FSDP（Fully Sharded Data Parallelism）**。FSDP 就是 ZeRO-3 的 PyTorch 原生实现：把模型参数、梯度、optimizer states 全部切分到各 GPU，使用时按需收集。

但 FSDP 有两个关键差异：

第一，**FSDP 是 PyTorch 核心的一部分**。不需要额外依赖，不需要修改模型定义，只需在模型外套一个 `FullyShardedDataParallel` wrapper。对于不想引入 DeepSpeed 的项目来说，FSDP 是"开箱即用"的 ZeRO-3 体验。

第二，**FSDP 比 ZeRO-3 更灵活**。ZeRO-3 的通信模式是固定的（统一切分，统一收集），FSDP 支持多种 sharding 策略——从 no sharding（等效于 DP）到 full shard（等效于 ZeRO-3）再到 hybrid shard（节点内保留完整参数，节点间切分）。这种灵活性让 FSDP 在不同规模的模型中都能找到合适的配置。

FSDP 的发布恰逢其时。2023 年 Meta 发布了 **LLaMA 系列模型**——13B、33B、65B。这些模型不是用 FSDP 训练的（Meta 内部用的是定制并行方案），但 LLaMA 开源后，社区立刻需要一套干净的训练方案。FSDP 正好成为 LLaMA 微调的标准工具。LLaMA 65B + 单机 8 张 A100 + FSDP——这套组合拳降低了"训大模型"的门槛，让更多学术实验室和小型团队可以训练 10B+ 量级的模型。

2024 年 2 月，Meta 发布了 **LLaMA 3 405B** 的训练细节。关键信息：405B 模型在 16384 张 H100 80GB 上训练，用了 FSDP 的定制版本（配合了一些 Meta 内部的优化）。这是 FSDP 在最大规模上的验证——证明 FSDP 可以扩展到

千卡级别。

但 FSDP 也有边界。当模型大到 100B 以上，FSDP 的通信开销开始影响效率。FP16 下 100B 模型每次 all-gather 需要传输 ~200GB 的数据（FP16 模型参数 + 梯度）。FSDP 的默认实现跟 ZeRO-3 一样，使用同步通信——通信等待时间在千卡集群上可能达到数十秒。Meta 的解决方案是 async all-gather（异步通信），让通信和计算重叠——这是一个不太优雅但有效的 hack。

FSDP 和 DeepSpeed 的关系在 2024 年演变为**两条路线**：

- 你要训 70B 以下模型，团队规模小、不想折腾 → **FSDP**（零依赖，开箱即用）
- 你要训 100B+ 模型，有专门的 infra 团队 → **DeepSpeed ZeRO + Megatron**（极致性能，但配置更复杂）

这个格局在 2024 年基本确定，2025-2026 年没有太大变化。

3.6 DeepSeek 的效率奇迹（2024-2025）

2024 年 12 月，DeepSeek 发布 **DeepSeek-V3**——671B MoE（37B 激活参数），训练成本 $5.57M。这个数字在发布时没有立刻引起轰动，因为它看起来只是一个"又大又便宜的模型"。但当行业仔细分析这篇论文时，发现它包含三项突破性的系统优化。

第一，FP8 混合精度训练。

在 DeepSeek-V3 之前，FP8 训练被视为"实验性的"——精度风险太大，没有人敢在 100B+ 规模上全量使用。DeepSeek 做了三件事让 FP8 训练在 2048 H800 上稳定运行：

- block-wise quantization：每个 1×128 的 tile 单独计算 scale 和 offset，比 layer-wise 量化更精细
- FP8 GEMM 累加精度保持 FP32：在矩阵乘法的累加阶段保持 32 bit 精度，避免 FP8 的精度溢出
- master weights 保持 FP32：每 N 步用一个单独的 FP32 副本做一次参数校正，防止精度漂移

这三点都不是创新性的——每个做过量化训练的人都知道这些技巧。DeepSeek 的创新在于**把它们组合在一起，并在 2000+ GPU 集群上验证了稳定性和精度**。在 DeepSeek 之前，没有任何开源工作证明过千卡级 FP8 训练的可行性。

第二，DualPipe 通信重叠。

DeepSeek-V3 的 MoE 架构带来了 all-to-all 通信——token 需要在不同 GPU 上的 expert 之间跳转。在 MoE 训练中，每个 Transformer 层的 MoE block 需要一次 all-to-all 通信来 dispatch token 到对应的 expert，然后再一次 all-to-all 来 combine 结果。这些通信在 GPU 的 flops 计算中插入了闲置等待。

DualPipe 的解决方案：把 forward 计算和 backward 计算跟 all-to-all 通信重叠。具体来说，DualPipe 在计算一个 micro-batch 的 MoE block 时，同时启动下一个 micro-batch 的 all-to-all 通信。这跟 CPU 的指令流水线异曲同工——当计算单元在算这一批的时候，通信单元在传下一批的数据。

效果：DualPipe 把 MoE 训练中的 all-to-all 通信延迟几乎完全隐藏在计算后面。通信空闲时间从 ~30% 降到 ~5%。这个优化对高通信占比的 MoE 训练来说，等效于白送了 25% 的总计算能力。

第三，aux-loss-free 负载均衡。

前面提到，MoE 训练需要一个 auxiliary loss 来防止所有 token 涌入同一个 expert。这个 aux loss 的权重需要调参——权重太大，模型质量下降；权重太小，负载不均衡，部分 expert 闲置。DeepSeek 发现这个问题的核心是一个**通信问题**：负载不均衡导致某些 GPU 的 expert 计算压力过大，拖慢全局。

DeepSeek 的解决方案是动态调整每个 expert 的路由 bias 参数。不需要辅助 loss，不需要调参。跟 DualPipe 结合后，不需要为负载均衡等待。

这三项优化叠加的效果是惊人的。DeepSeek-V3 的 MFU 达到了 **~70%**——相比之下，2024 年的行业平均 MFU 约为 35-40%。DeepSeek 用同样的 2048 块 H800，比行业平均水平多算了一倍的 flops。这才是 $5.57M 训练成本的核心来源——不是硬件便宜，是用得更高效。

DeepSeek-V3 让整个行业开始重新思考一个问题：**训练效率的瓶颈到底在哪里？** 在 DeepSeek 之前，共识是"训练效率受限于硬件，GPU 不够快，HBM 带宽不够宽"。DeepSeek 证明了系统优化的天花板远高于硬件天花板——~70% 的 MFU 说明还有 30% 的优化空间，而行业平均 35% 的 MFU 说明大多数训练系统有 3-4 倍的优化余地。

2025 年初，DeepSeek 发布了 R1（推理模型），论文中提到训练 R1 时使用了类似的系统优化加上了 RL 训练阶段的优化。DeepSeek 被行业称为"AI 界的拼多多"——不是因为它便宜，而是因为它用极致的系统效率把成本和进度的等式重新定义了。

3.7 IO 觉醒（2022-2024）：FlashAttention 改写 Attention 的底层逻辑

如果说 ZeRO 改变了显存管理的范式，FlashAttention 改变了 Attention 计算的范式。它可能是 2020 年代以来**单项影响力最大**的训练优化技术。

2022 年 5 月，Stanford 博士生 Tri Dao 和 Christopher Re 团队提交了 FlashAttention 论文，被 NeurIPS 2022 接收。 论文的核心洞察极其根本：Attention 的瓶颈不是算力（FLOPS），而是显存带宽（memory bandwidth）。

标准 Attention 实现需要计算一个 N×N 的注意力分数矩阵，把它完整地写出到 HBM（高带宽显存），再从 HBM 读回来计算 softmax——两次 HBM 访问，每次 $O(N^2)$。当 N=4096 时，这个矩阵是 16M 个元素；当 N=128K 时，它是 16B 个元素——已经超过了大部分 GPU 的 HBM 容量。

FlashAttention 的解决方案：通过 tiling（分块），在 GPU 的 SRAM（高速缓存）上完成 Attention 的子计算，只把最终结果写回 HBM。**HBM 读写从 $O(N^2)$ 降到 $O(N)$。** 同时通过不保存完整的 Attention 矩阵（反向时重新计算），进一步节省显存。实验数据：BERT 训练提速 15%，GPT-2 训练提速 3×。

2023 年 7 月，FlashAttention 2 发布。 没有引入新的算法思想，而是做了更精细的工程优化：更好的线程块并行分配、减少非 matmul 操作、更高效的 causal masking。比 FA1 再快约 2×，达到 A100 FP16/BF16 理论峰值的 50-73%。

2024 年 7 月，FlashAttention 3 发布。 这次是 NVIDIA 的 Jay Shah 和 Tri Dao 的联合成果。FA3 深度利用 Hopper H100 的新硬件特性——FP8 Tensor Core 和异步 WGMMA 指令——在 FP16 下达到了 740 TFLOPS，在 FP8 下达到了 1.2 PFLOPs。

FA1→2→3 的演进揭示了一个更深的规律：**AI 框架的创新正在从"纯算法创新"转向"硬件-软件协同设计"。** 每一代新硬件都打开了新的优化空间，但代价是代码的可移植性降低——FA3 的核心优化在 A100 上无法使用。

3.8 硬件基础设施的演进

训练框架不能脱离硬件单独讨论。以下硬件层面的演进直接塑造了框架的设计方向。

NVIDIA DGX 系列的代际进化：

代际	发布	GPU	NVLink 带宽	对框架的意义
DGX-1	2016	8×P100	150 GB/s	TP 还不可行
DGX-2	2018	16×V100	300 GB/s	NVSwitch 引入
DGX A100	2020	8×A100	600 GB/s	TP 变得可行
DGX H100	2023	8×H100	900 GB/s	TP 成为标配
DGX B200	2025	8×B200	1.8 TB/s	FP8 训练成熟

其中 DGX A100 的 NVSwitch 架构是一个关键转折：它让节点内 8 张卡之间的通信不再需要经过 CPU 或网卡，而是通过 NVSwitch 全互联。这使得 TP 在 8 卡节点内的 AllReduce 通信延迟降低到可接受的范围。

HBM 容量 vs 模型规模的赛跑：

年份	代表性 GPU	HBM	代表性模型	参数
2018	V100 32GB	HBM2	BERT	340M
2020	A100 80GB	HBM2e	GPT-3	175B
2023	H100 80GB	HBM3	Llama 3	405B
2025	B200 141GB	HBM3e	—	~1T?

HBM 容量在 7 年里只增长了 4.4 倍，而同一时期的典型模型参数增长了 1,200 倍。这个剪刀差是 ZeRO 和 Offload 等技术存在的根本原因。

四、横向分析：2026 年的训练格局

4.1 四大训练框架的选型

六年下来，四个主流框架形成了清晰的生态位：

维度	DeepSpeed	Megatron-LM	FSDP	JAX
维护方	微软	NVIDIA	PyTorch / Meta	Google
开源协议	Apache 2.0	自定义（含闭源组件）	BSD	Apache 2.0
核心能力	ZeRO-1/2/3, MoE 原生支持	TP, PP, EP, 极致性能	ZeRO-3 仿制, hybrid shard	XLA 编译, pmap/pjit
易用性	中等（配置项多）	低（需要手动切分模型）	高（一行 wrapper）	中高（函数式 API）
极致性能	优	最佳	良	优（在 TPU 上）
MoE 支持	原生支持	支持	有限（需要定制）	支持
适用规模	1B-1000B+	10B-1000B+	1B-100B	1B-1000B+
GPU 支持	NVIDIA/AMD	NVIDIA only	NVIDIA/AMD/Intel	TPU / NVIDIA
社区活跃度	高	中	高	中

选型本质不是技术问题，而是规模问题：

- 如果你训 <10B 模型 + 团队没有专职 infra → **FSDP**。它帮你多快好省地完成 80% 的任务。
- 如果你训 10B-100B + 对性能有较高要求 → **DeepSpeed ZeRO-3**。配置比 FSDP 复杂，但在显存节省和通信模式上有更多控制。
- 如果你训 100B+ → **DeepSpeed + Megatron 的组合**（3D 并行）。没有选择。FSDP 在这个规模下通信效率不够，Megatron 的 TP/PP 是目前唯一经过验证的方案。
- 如果你在 Google Cloud 用 TPU → **JAX**。不选 JAX 等于浪费 TPU 的性能。

2025-2026 年的趋势是框架界限开始模糊。DeepSpeed 加入了 TP 支持的实验性功能，FSDP 开始借鉴 Megatron 的通信优化，PyTorch 的 DDP（Distributed DataParallel）也在吸收 ZeRO 的思想。框架竞争正在从"要不要用我"变成"谁能提供更透明的底层抽象"——当 `torch.distributed` 的原语足够丰富，很多框架层的工作就变得不那么必要了。

4.2 五个核心框架深度分析

DeepSpeed（微软）——功能最全的"瑞士军刀"

DeepSpeed 的核心优势不是某一个功能——是**功能的广度**。ZeRO 三阶段 + Offload + MoE + RLHF 优化 + 通信压缩 + 推理优化，几乎覆盖了训练全流程。

ZeRO 系列是 DeepSpeed 安身立命的根本。ZeRO-Offload 和 ZeRO-Infinity 是目前唯一成熟的显存卸载方案——FSDP 没有 Offload，因此在显存受限的场景（单卡/少卡训练）下，DeepSpeed 几乎是唯一选择。

DeepSpeed-Chat（2023 年 3 月） 把 RLHF 训练从大厂的专利变成了可负担的开源方案。在 8×A100-80G 上 9 小时训练 13B 模型，成本 $290。

DeepSpeed-MoE（2024 年 3 月） 在 ZeRO 上增加了 Expert Parallelism，是生产环境最早可用的 MoE 训练方案。

与 HuggingFace Transformers 的深度集成是 DeepSpeed 在中小企业市场的护城河。使用 Transformers + DeepSpeed 是最流行的"开箱即用"大模型训练方案。

劣势： 代码复杂度高，API 变化频繁；说到底，是"套在 PyTorch 外面的层"，跟 PyTorch 新特性的兼容性有时滞后。

Megatron-LM / Megatron Core（NVIDIA）——MFU 之王

Megatron 走的是一条跟 DeepSpeed 截然不同的路——**深度绑定 NVIDIA 硬件，追求极致利用率**。H100 上 FP8 训练的 MFU 达到 42-50%，是所有框架中最高的。

Tensor Parallelism 的原创者。 没有 Megatron 的 TP，3D 并行就不存在。Megatron Core（2024 重构）把 TP 做成了模块化组件，支持更灵活的并行组合。**Pipeline Parallelism 的 1F1B 调度**在气泡开销上远小于 GPipe 的同步调度方案。

劣势： 生态闭环——只支持 NVIDIA GPU；代码复杂度是所有框架中最高的；2024 年之前的开源滞后策略也限制了社区采用。

FSDP / PyTorch Distributed（Meta）——学起来最舒服的

FSDP 的设计哲学是"用最简单的方式做最核心的事"。它不做 Offload、不做 MoE EP、不做通信压缩，但把 ZeRO Stage 3 的功能跟 PyTorch 生态无缝融合。

跟 PyTorch 的原生集成是 FSDP 最强的护城河。FSDP 跟 `torch.compile`、DTensor、`torch.distributed.pipelining` 的协作是其他框架不能比的。

Llama 系列的大规模验证。 Llama 3 405B（30,720 H100）是 FSDP 的上限测试——证明了 FSDP 可以扩展到 3 万张 GPU 的规模。

劣势： 没有 Offload、没有 MoE EP、超大规模深度优化不如 Megatron-DeepSpeed。存在一个有趣的矛盾：**FSDP 的"用户"最多**，但**真正用 FSDP 大规模训练大模型的比例并不高**——大厂内部通常深度定制，中小团队更多用 HuggingFace + DeepSpeed。

从社区实践来看，选择 FSDP 的主要场景是：① 不想引入 DeepSpeed 依赖的 PyTorch 项目；② 正在使用 torch.compile 等 PyTorch 新特性；③ 模型规模在 10B-70B 之间，不需要 Offload 和 MoE。选择 DeepSpeed 的场景是：① 模型超过 70B；② 需要 Offload；③ 需要 MoE 支持。

ColossalAI（潞晨科技）——自动化的探索者

ColossalAI 的差异化在于**降低门槛**。用户只需要加一行 `@colossalai.autoparallel`，框架自动在策略空间中搜索最优的并行组合。**ColossalAI Gemini** 对标 ZeRO，支持 GPU + CPU + NVMe 三级存储的异构内存管理。

现状： ~41k Stars 在开源社区有不小的影响力，但实际大规模训练部署远少于 DeepSpeed 和 Megatron。自动并行的理念很好，但"自动"在大规模场景下依然不如"专家手动调优"。

JAX + PaxML（Google）——TPU 的独有语言

JAX 走了一条完全不同的路：**函数式编程 + XLA 编译 + TPU 原生支持。** `jit`（即时编译）、`vmap`（向量化映射）、`pmap`（并行映射）等函数变换表达并行计算。PaLM 和 Gemini 系列都用的是 JAX + Pathways。

劣势极其明显： 跟 PyTorch 生态不兼容，学习曲线陡峭，TPU 绑定了训练平台（Google Cloud 用户才能用好）。

关键对比维度

维度	DeepSpeed	Megatron Core	FSDP	ColossalAI	JAX
并行策略	ZeRO+TP+PP+EP	TP+PP+SP+EP	ZeRO(FSDP)	自动并行	pmap+SPMD
FP8 训练	✅	✅ (最佳)	❌	✅	✅ (TPU)
MoE EP	✅	✅	❌	✅	✅
RLHF 优化	✅ 最佳	❌	❌	✅	❌
Offload 卸载	✅ 最佳	❌	❌	✅ (Gemini)	❌
长上下文	Ulysses	自研 SP	社区方案	Ring Attn	自研
硬件绑定	通用 GPU	NVIDIA only	通用 GPU	通用 GPU	TPU 优先
易用性	中等	低	高	高	极低
MFU 典型值	35–45%	42–50%	30–40%	30–40%	40–50%(TPU)

用户群分布

- **Meta（Llama 系列）**：FSDP 定制。Llama 3 405B 在 30,720 H100 上训练 54 天，MFU ~38–43%
- **Microsoft（Phi、Copilot）**：DeepSpeed。自家框架深度优化
- **NVIDIA（Nemotron）**：Megatron Core。软硬一体验证平台
- **Google（PaLM、Gemini）**：JAX + Pathways。完全自研
- **OpenAI（GPT-4、o1）**：自研框架，与 Azure 深度绑定——具体未公开
- **DeepSeek**：自研 HAI-LLM（基于 PyTorch）。FP8 训练先行者，MFU ~50%+
- **Mistral、xAI**：Megatron-DeepSpeed 或 FSDP
- **学术/创业团队**：FSDP（首选，门槛低）或 ColossalAI（自动并行）

4.3 并行策略的选择图谱

2026 年，并行策略的选择已经有了清晰的边界条件：

模型规模	推荐策略	通信瓶颈
<1B	DP only, 单节点	无
1B–7B	DeepSpeed ZeRO-2/3 或 FSDP, 1–4 GPU	all-reduce 带宽
7B–70B	FSDP 或 DeepSpeed ZeRO-3, 8–64 GPU	跨节点 all-gather
70B–400B Dense	TP=4/8 + PP=2/4 + DP (Megatron + DeepSpeed), 64–1024 GPU	NVLink + 跨节点带宽
MoE >100B	上述 + EP (Expert Parallelism), 通信优化优先	all-to-all 通信

选型的不确定性在高并发 MoE 训练场景下最大。因为 EP 的通信模式（all-to-all）跟 DP（all-reduce）和 TP（all-reduce 但更密集且更依赖 NVLink）完全不同。一个 all-to-all 操作中，每张 GPU 要同时向其他所有 GPU 发送数据和接收数据。当 GPU 数量很大时（2048+），all-to-all 的通信拓扑复杂度指数级增长。

一个常被忽略的问题：**并行策略的"维度选择"本质是"算力 ÷ 通信带宽"的函数**。不同 GPU 的算力和带宽比值不同：

- H100 SXM：989 TFLOPS FP8 / 3.35 TB/s HBM3 ≈ 295 FLOPS/byte
- MI300X：1307 TFLOPS FP8 / 5.2 TB/s HBM3 ≈ 251 FLOPS/byte
- TPUv5p：459 TFLOPS BF16 / 未知，TPU 互联带宽 ~4.8 TB/s

FLOPS/byte 比值越大，说明计算相对通信更便宜，很适合 TP（计算密集但通信密集）。比值越小，说明通信相对更贵，应该优先考虑 DP（通信量小但计算压力分散）。这个比值在不同硬件上差异显著，意味着**同样的 70B 模型在 H100 和 MI300X 上应该选择不同的并行策略配置**。

4.4 通信：分布式训练的真正瓶颈

分布式训练中通信的占比随着 GPU 数量增加而增长。这是所有大规模训练团队都会面对的问题。

主要通信模式及成本：

All-reduce（DP 和 ZeRO 的梯度同步）：每步需要传输的总数据量 = model_params × precision_bytes × (N_gpu − 1) / N_gpu。对于 70B 模型 FP16，每步 all-reduce 传输 ~140GB 数据。在 8 张 H100 (NVLink 900GB/s) 上耗时约 15ms；在跨节点 (100Gbps Ethernet) 上耗时约 100ms。

All-gather + Reduce-scatter（ZeRO-3 的参数收集和梯度分散）：ZeRO-3 在 forward 时做 all-gather 收集参数，在 backward 时做 reduce-scatter 分散梯度。通信量跟 all-reduce 相似，但通信模式不同——all-gather 是"每张卡都要把持有的一部分数据发给所有人"，reduce-scatter 是"每个人把一部分数据汇总后分散给不同卡"。ZeRO-3 的通信模式相比标准 all-reduce 有额外 ~20% 的开销，但显存节省了 16x+。

All-to-all（MoE 的 Expert 分发）：每张 GPU 把 token 发给所有其他 GPU 上对应的 expert。通信量 = tokens_per_gpu × hidden_dim × precision_bytes × (n_experts − 1) / n_experts × topk。对于 DeepSeek-V3 的配置（37B 激活，top-8 routing，2048 GPUs），每次 all-to-all 通信约传输 1–2GB 数据，耗时 ~5–10ms。看起来不大，但每个 Transformer 层的 MoE block 都需要两次 all-to-all，叠加上去就很可观了。

解决通信瓶颈的四种方案：

1. **通信与计算重叠**（异步通信）。DP 的 all-reduce 可以和下一层的 forward 计算重叠——GPU 在算 FFN 的时候，通信单元在同步上一层的梯度。ZeRO-3 的 all-gather 也可以跟当前的 forward 计算重叠。这个方案不增加成本，只需要在框架层把计算 kernel 和通信 kernel 异步化。PyTorch 2.x 的 `torch.distributed.all_reduce` 已经原生支持 async 模式，但实际使用中需要精细的调度安排——不是简单地加一个 `async_op=True` 就万事大吉。

1. **通信压缩**。梯度压缩（1-bit SGD、TopK sparsification）在训练时可以减少 99% 的通信量，但会影响收敛。1-bit Adam（2021, Tang et al.）通过把梯度量化为 1-bit（符号位）来通信，把梯度同步的带宽需求降低 16x——理论上可以在廉价以太网上做千卡级 DP。问题是对 Adam 优化器的修改较多，不是所有模型都能直接用。

1. **拓扑感知的切分策略**。让紧密通信（TP）走 NVLink，稀疏通信（DP、PP）走 Ethernet。这个方案是目前大厂的标准做法——NVIDIA 的 DGX SuperPOD 就是按这个思路设计的（8 GPU in NVSwitch domain → 多个 domain via InfiniBand）。但云环境（租用 GPU 实例）无法保证拓扑感知——你租的 8 张 A100 可能在同一个节点也可能分散在不同节点。这就是为什么 3D 并行在云环境中的部署比裸金属集群更加不可控。

1. **DualPipe（DeepSeek 的方案）**。在 step 级别而不是 kernel 级别做通信-计算重叠。DualPipe 把一个 micro-batch 的 backward 和下一个 micro-batch 的 forward 计算与 all-to-all 通信重叠。这个方案的侵入性很大——需要完全掌握模型的计算图，在框架最底层改写调度逻辑。DeepSeek 能做到是因为他们在从头搭建训练框架，而大多数团队用的是现成的 DeepSpeed/Megatron 组合。

4.5 训练成本的急剧下降

过去五年，训练成本的下降是线性的——不是摩尔定律式的指数下降，但考虑到模型规模同期增长了两个数量级，每 token 训练成本下降的速度其实很惊人。

2020 年：GPT-3 175B 训练成本约 $12M（估计）。基于 OpenAI 披露的算力和时间推算：数千张 V100 运行数周，加上研发投入和调试成本。注意这只是训练一次的裸成本——不包括探索性实验、失败的训练 run、超参数搜索。

2022 年：PaLM 540B 训练成本约 $10-20M。Google 使用自家 TPUv4，算力成本比 NVIDIA GPU 低——但模型更大（540B vs 175B），训练时间更长。PaLM 的论文没有公布训练成本，行业估算在千万美元级别。

2023 年：LLaMA 65B 训练成本约 $2M。这是开源模型的里程碑——Meta 用 $2M 训练的开源模型达到了接近 GPT-3.5 的水平。此时行业出现了一个关键认知转变：**训练成本降低的主要驱动力不是硬件降价，而是系统效率提升和技术进步**。LLaMA 65B 在 2048 A100 80GB 上训练了 21 天，等效算力利用效率显著优于 GPT-3 时代。

2024 年 1 月：DeepSeek-V2 训练成本 ~$6M（236B MoE）。FP8 混合精度训练的早期尝试。当时行业认为这已经是很低的成本了——跟 LLaMA 65B 的 $2M 相比，DeepSeek-V2 大了近 4 倍但成本只有 3 倍。

2024 年 12 月：DeepSeek-V3 训练成本 $5.57M（671B MoE, 37B 激活）。这个数字让全行业震惊——671B 的"总参数"模型只花了 $5.57M，比很多 70B Dense 模型的训练还便宜。核心原因：MoE 的激活参数只有 37B（训练的计算成本跟 37B Dense 模型差不多），加上~70% 的 MFU，以及 FP8 训练的低成本。

2024-2025 年：LLaMA 3 405B 训练成本约 $50M+（行业估计）。405B Dense，在 16384 H100 上训练，数据量 15T+ tokens。这是 2026 年已知的最大训练成本。

数字背后的驱动力：

驱动力	效果
FP16→FP8 量化训练	每 token 计算成本降低 30-50%（相同吞吐，更低的精度）
系统效率提升	MFU 从 35% 到 70%，等效于 2x GPU 利用率
MoE 架构	相同总参数下训练成本仅为 Dense 的 1/5-1/10
硬件代际提升	H100→B200 每代每 token 成本降低 20-30%
数据效率	更好的数据清洗和混合策略，减少无效训练步数

但一个趋势值得关注：**绝对训练成本正在触底反弹**。2020-2024 年成本快速下降的效率红利正在被更大的模型和更长的训练消耗掉。DeepSeek-V3 的 $5.57M 让行业看到了"极限条件下的最低成本"，但接下来的模型（GPT-5、Claude 4、Gemini 3）的训练成本估计将重回 $100M+ 级别——不是因为效率倒退，而是因为模型更大、数据更多、训练更长。

4.6 MoE 训练 vs Dense 训练

MoE（Mixture of Experts）在 2024-2026 年成为业界主流。几乎所有新发布的大模型——DeepSeek-V3、Mixtral 8x22B、Qwen2.5-MoE——都用了 MoE 架构。Dense 模型只剩 LLaMA 3 405B 和 Google Gemini 的部分变体继续坚持。

MoE 的优势很明显：

- **更多参数，相同计算量**。MoE 的核心是"总参数可以很大，但每次推理/训练只激活一部分"。DeepSeek-V3 671B 总参数但只有 37B 激活，训练的计算成本跟 37B Dense 模型相当，但模型的知识容量是 671B 级的。
- **专业化的 expert**。理论上，不同的 expert 可以发展出不同的"专长"——有的擅长推理，有的擅长代码，有的擅长长文本。但实际上 expert 的专业化程度取决于路由机制的质量——如果路由分布不均匀，所有 token 都涌向同一个 expert，MoE 就退化成了 Dense。

MoE 训练面临的挑战同样显著：

1. **负载均衡**。MoE 的经典问题：路由机制倾向于把 token 发给同一批"强 expert"。如果不加干预，大部分 expert 都处于半闲置状态。解决方案经历了三个代际：auxiliary loss（降质量）→ 动态 bias 调整（DeepSeek, 2024, 不降质量但计算额外开销）→ expert dropping（极端情况丢弃部分 token 的计算）。负载均衡在 MoE 训练中不是"锦上添花"，而是"不解决就训不动"的硬约束。

1. **All-to-all 通信开销**。MoE 训练的通信模式跟 Dense 完全不同。Dense 的 DP 通信是 all-reduce（一对多/多对一模式，可以跟计算重叠）。MoE 的 EP 通信是 all-to-all（每张卡要向所有其他卡发数据，也要收来自所有其他卡的数据）。all-to-all 的通信量跟 GPU 数量线性增长，所以在千卡级 MoE 训练中，all-to-all 很快成为瓶颈。

1. **训练不稳定**。MoE 训练比 Dense 训练更容易出现 loss spike（损失值突然飙升）。常见原因：某个 expert 的参数更新过大，导致 routng 突然偏向或偏离该 expert。解决方案包括 gradient clipping（更激进的梯度裁剪）和 expert-level gradient normalization。

1. **总显存消耗大**。MoE 的总参数虽然只有部分被激活，但所有专家的参数都必须保存在显存中（训练时反向传播需要更新所有参数）。671B FP16 模型需要 ~1.3TB 显存来存储参数（即使激活参数只有 37B）。ZeRO-3 + EP 的组合让

这成为可能——但配置复杂度远超同等激活参数的 Dense 模型。

训练成本对比（2026 年数据）：

维度	Dense	MoE
模型容量 vs 总参数	线性相关	总参数 5–20x 激活参数
同质量训练成本	基线	约 1/5–1/10
硬件需求（显存）	低	高（需要更多 GPU 或更深的 ZeRO）
通信模式	all-reduce（可重叠）	all-to-all（难重叠，DualPipe 可缓解）
推理成本	基线	约 1/5–1/10（同质量下）
训练稳定性	好	较差，需要更多调参
框架支持成熟度	成熟（所有框架）	DeepSpeed/Megatron 成熟，FSDP 有限

行业共识：2026 年训新模型，MoE 是默认选择。Dense 只在两种情况下使用：模型 <10B，MoE 的 overhead 不值得；或者团队没有能力处理 MoE 训练的稳定性问题。

4.7 训练云平台

框架之上还有更上层的云平台。这层直接影响框架的选择——不同云平台对不同框架的优化深度不同。

NVIDIA DGX Cloud：预装 Megatron Core 的端到端训练服务。NVIDIA 提供从集群规划、网络拓扑到框架调优的全套服务。硬件稀缺性是最大的壁垒：B200 集群不是谁都能排队买到的。

AWS SageMaker：支持 DeepSpeed 和 FSDP，跟 S3/EFS 存储生态深度集成。优势是企业级合规和全球部署。

Google Cloud TPU + GKE：JAX/Pathways 的唯一云平台。TPU v5p 和 Trillium（TPU v6）算力密度极高——单 Pod 超 8,000 芯片，但灵活性差：TPU 只适合训练，推理、微调、数据预处理都需要另外的方案。

Azure ND 系列：微软自家的 GPU 云，跟 DeepSpeed 做了最深的联合优化。GPT-4 和 Copilot 模型都在 Azure NDv5（H100）集群上训练。

CoreWeave / Lambda Labs / RunPod：专业 AI 云，只做 GPU 算力。核心优势是"GPU 就是产品"——没有传统云厂商的虚拟机 overhead，价格通常比三大云便宜 30–50%。

选择云平台，说到底是在做一道权衡题：部署深度（DGX Cloud 的 Megatron 优化最佳）vs 生态宽度（AWS 的集成最全）vs 成本（CoreWeave 最便宜）vs 合规（Azure 的企业安全最好）。框架的选择往往不是独立的——选了什么云，往往就决定了默认用什么框架。

五、横纵交汇洞察

5.1 训练工程的核心矛盾

六年来的每一个重大突破，都是在一个核心矛盾中寻找更好的平衡点：**模型更大 = 更聪明，但训练太贵 = 训不起**。

这个问题在 2020 年 GPT-3 发布时第一次成为焦点——GPT-3 的论文证明了 scaling law（模型越大，损失越低），但 $12M 的训练成本让人望而却步。于是整个行业开始回答一个问题：怎么在"更大"和"更贵"之间找到更有利的折中?

每个突破性的工作都从这个角度重写了等式：

- **Megatron-LM**（2020）：模型做大不可怕，把层切开就行了。代价是增加了通信复杂度，但相比模型做不大的损失，这个代价是值得的。
- **ZeRO**（2020）：你不需要那么多显存去存冗余的 optimizer states。解决了"模型能塞进 GPU"的问题——不增加通信复杂度（ZeRO-1/2）或少量增加（ZeRO-3）。
- **MoE**（2021-2024）：让模型参数变大，但每一步的计算量只跟激活参数有关。以通信复杂度（all-to-all）和训练稳定性（负载均衡）为代价，换来了 5-10x 的性价比提升。
- **DeepSeek-V3**（2024）：FP8 训练 + DualPipe + 负载均衡优化，把 MFU 推到了 ~70%。DeepSeek 证明了**算法创新比硬件进步更能改变成本曲线**——同样 2048 块 H800，他们的等效算力是行业平均的两倍。

每一个突破都没有消除矛盾，只是把矛盾转移到了另一个维度。Megatron 把"放不下"变成了"通信太慢"，ZeRO 把"显存不够"变成了"通信更多"，MoE 把"模型太大"变成了"训练不稳定"。**分布式训练的本质就是不断地把问题从一个约束转移到另一个约束**，直到你找到最优的落点。

5.2 效率提升的四个来源

过去五年的训练效率提升来自四个独立维度：

来源	典型进展	效果（五年累计）
硬件进步	V100 → A100 → H100 → B200; HBM 带宽提升 5x; NVLink 带宽提升 3x	算力增长 ~10x
算法创新	FlashAttention, MLA, FP8 训练, MoE 架构	同等模型规模下算力需求降至 1/5–1/10
系统优化	ZeRO, DualPipe, async communication, 3D parallelism	MFU 从 ~30% 提升到 ~70%
数据效率	更好的数据混合策略, 减少无效 token 训练, 训练数据质量筛选	同质量下训练步数减少 30–50%

四个维度的效果不是简单叠加——它们是相乘的。假设硬件进步 10x（同等功耗下算力提升），算法创新把算力需求降到了 1/5，系统优化再把 MFU 从 30% 提升到 70%（2x），数据效率再省 30% 的步数。组合效应就是：$10 \times 5 \times 2 \times 1.3 \approx 130$x 的等效效率提升。这不精确（维度之间有相关性，不是完全独立的乘积），但方向是对的。

四个维度的相对贡献正在发生微妙变化：

- **硬件的收益在递减**。H100 到 B200 的算力提升约 30–50%，远不如 V100 到 A100（~3x）和 A100 到 H100（~6x）。GPU 性能提升正在逼近物理极限（功耗、散热、晶体管密度）。

- **算法的收益在加速**。MoE + FP8 的组合在 2024 年让 DeepSeek-V3 实现了 10x 性价比优势。这是过去几年最大的一次算法跳跃。
- **系统的收益在持续**。DeepSeek 证明了 MFU 还有 2x 的提升空间（行业平均 35% → 极限 ~70%）。系统优化的天花板不是硬件，是工程投入。
- **数据的收益开始显现**。当模型和数据都大到一定程度，"训更多"的收益在递减，"训更好"的收益在增加。数据清洗、去重、混合策略的质量对训练结果的差异比模型架构更大。

趋势：**2026-2028 年训练效率提升的主要驱动力将从硬件转向算法和系统**。硬件增长放缓，但算法创新（更高效的架构、更高效的训练范式）和系统优化（更高的 MFU、更智能的调度）的潜力远未耗尽。

5.3 历史如何塑造了今天的竞争格局

把纵向线和横向线叠在一起看，最突出的特征是：**今天的框架格局是 2019-2020 年"双星闪耀"的直接产物。** 几乎所有后续的发展——FSDP 的分叉、3D 并行的确立、ColossalAI 的入场——都可以追溯到 Megatron-LM 和 ZeRO 这两篇论文。

DeepSpeed 和 Megatron-LM 在几乎同一时间提出了两种互补的方案——一个消除冗余（ZeRO），一个切分计算（TP）。两者的融合（Megatron-DeepSpeed，2021）至今是超大规模训练的技术基础。

FSDP 的诞生（2022 年 3 月）是唯一的重大格局变动。但不是因为技术——是因为**战略**。Meta 不想让 PyTorch 的分布式训练能力依赖微软。这个"分叉"虽然带来了两套 API 的社区分裂，但也催生了良性竞争。

优势与劣势的历史根源：

- **DeepSpeed 的核心优势——功能的广度——根植于 2020 年的激进扩张策略。** 一年内连发三个 ZeRO 版本，这种速度在后来的竞争中没有被复制。
- **Megatron Core 的极致 MFU——根植于 NVIDIA 的"软硬一体"路线。** 其他框架等硬件出了再适配，Megatron 在新硬件还没上市时就做预研。
- **FSDP 的局限（没有 Offload）——根植于"简洁优先"的设计哲学。** Offload 涉及复杂的 CPU-GPU 调度和 PCIe 带宽管理，Meta 选择不做以保持 API 简洁。
- **JAX 的局限（PyTorch 不兼容）——根植于 Google 在框架战争中的失败。** TensorFlow 被 PyTorch 击败后，Google 选择了另辟蹊径。这不是技术选择，而是战略收缩。

5.4 被抛弃的路线

回顾发展史，有几个方向在竞争中边缘化了：

- **纯 Pipeline Parallelism（GPipe 等）**：单用 PP 效率低（气泡问题），必须和 TP/DP 组合使用
- **Google Mesh TensorFlow / GShard**：跟 PyTorch 生态不兼容，影响力局限在 Google 内部
- **手动 Model Parallelism**：手工切分代码维护成本高，已让位于自动化的 TP/PP
- **纯 Data Parallelism（无 ZeRO）**：模型超过 1B 就不可用
- **单进程多线程 DataParallel**：GIL 瓶颈严重，PyTorch 已不再推荐

5.5 未来：训练会变成什么

预训练和后训练的融合

传统的"预训练 → 微调 → RLHF"三段式流水线正在模糊边界。2025-2026 年出现了几个趋势：

- **Continuous Training**：模型上线后继续训练（吸收新数据、修正旧错误）。DeepSeek 和 Google Gemini 的模型会在部署期间用新数据持续微调。这打破了"训练是一次性的"的假设——训练成了一个持续的过程。
- **RL-based Training**：o1/o3、DeepSeek-R1 的训练流程中，RL（强化学习）不再是"微调阶段"的一个步骤，而是训练的核心组成部分。RL 训练的计算量可能超过标准预训练。"训练"的语义由此扩展——不仅是语言建模的损失最小化，还包括 RL 策略的迭代优化。
- **Synthetic Data**：模型生成训练数据的循环——用当前最佳模型生成数据，训练下一代模型。DeepSeek-R1 和 GPT-5 的训练中都包含大量合成数据。这对训练基础设施提出了新的要求——你不仅要跑训练脚本，还要同时跑推理来生成训练数据。

训练时计算

2025 年出现的"推理时计算"（reasoning-time compute）在训练端的对应物是**训练时计算（training-time compute）的重新分配**。传统的训练是把所有 token 一视同仁——每个 token 贡献相同的 loss，每个 token 都被 100% 地学习。但有些 token 比其他 token 更重要（数学证明中的关键步骤、代码中的边界条件）。

2025-2026 年出现的研究方向：**adaptive compute allocation in training**——训练时对"困难 token"分配更多的计算资源。这跟 MoE 的思路一致，但更精细——在每个 token 级别控制分配的 flops 量。

这个方向的实践还很早期，但它代表了训练范式的可能转变：从"所有数据统一处理"到"根据数据重要性动态分配计算"。

SWARM 训练：跨数据中心、异构设备

2026 年最激进的方向之一是 SWARM 训练——让训练分布在不同地理位置、不同硬件上的设备上执行。

这个方向推翻了分布式训练的一个基本假设："所有 GPU 必须通过高速互联连接"。如果训练可以在公众的消费级 GPU 上执行（类似 SETI@home 的分布式计算模式），那么任何人都可以参与训练超大规模模型。

2025 年，Petals（一个去中心化的推理/训练平台）在 10B 模型的训练上初步验证了 SWARM 训练的可行性——在跨大陆的消费级 GPU 上实现了基本可用的训练吞吐。性能很差（跟专业集群比差 2-3 个数量级），但"能用"本身就是一个重大突破。

SWARM 训练面临的核心问题：网络延迟（跨洲延迟 100-200ms）、设备异构性（RTX 4090 + M2 Ultra + A100 混用）、可靠性（消费级设备随时离线）。这些问题在 2026 年还没有好的解决方案，但几个团队正在尝试。如果 SWARM 训练能再进步一个数量级，它对 AI 行业的影响可能比任何单一模型都大——因为它会打破"训练大模型只有科技巨头能做到"的垄断。

5.6 三个未来剧本

把技术趋势、历史格局、以及当前玩家的优劣势综合起来看，未来 3 年有三个可能的演进方向：

剧本一：生态整合，两强相争（60%）

DeepSpeed 和 Megatron Core 继续保持互补关系，FSDP 在中小规模场景扩大份额。2026-2027 年出现框架层标准的收敛——各家支持相同的并行策略组合，用户可以在 FSDP 和 DeepSpeed 之间透明切换。这个剧本最可能的原因是

大模型训练工具链已经非常成熟——再做一套新框架的边际收益很低。

剧本二：国产框架从"能用"到"好用"（25%）

华为昇腾 + 定制框架或 ColossalAI 在国产芯片上的适配，在出口管制加剧或国产芯片产能爆发时实现突破。国内大厂（字节跳动、阿里巴巴）加大在昇腾上的训练优化投入。

剧本三：范式切换颠覆现有框架（15%）

某种新的训练范式的出现——完全不需要传统反向传播的算法、神经拟态计算、或远超 Transformer 的新架构——让当前的并行策略积累失去价值。可能性不高，但回顾历史，AI 领域的"已知"往往只是阶段性认知。

5.7 国产框架的特殊处境

中国的大模型训练框架生态走了一条不同的路。DeepSeek 用的是自研 HAI-LLM，阿里通义千问用 PAI 平台（基于 Megatron 深度定制），字节跳动的 veTrain 是托管训练服务，百度文心系列用 PaddlePaddle Fleet，华为昇腾用 MindSpore。

大厂几乎都自研或深度魔改，不用开源框架的"原版"。 DeepSeek 的 HAI-LLM 虽然基于 DeepSpeed/Megatron，但做了大量定制，特别是 FP8 训练管线。阿里 PAI 平台也在 Megatron 上做了深度修改。

华为 MindSpore 是国产框架中最特殊的一个。 它走的是"软硬一体"路线——从昇腾芯片到 MindSpore 框架到 MindStudio 开发工具，全方位自研。2026 年的现状是：在昇腾 910B/910C 上跑训练，性能约为同级别 NVIDIA GPU 的 60-80%，兼容性问题仍然存在。

5.8 一条贯穿始终的暗线

整个训练框架发展史，说到底是一部**解耦与耦合的辩证关系史**：

ZeRO 把显存需求从"存全部"解耦为"存一份"，TP 又把多张卡的计算耦合在一起。FSDP 允许灵活配置哪些参数分片，Megatron Core 的 FP8 训练又把精度管理和计算管道紧密绑定。

更深层的是：**随着模型规模的指数级增长，我们越来越多地牺牲优雅性来换取可行性。** 一台 DGX H100 集群上运行训练任务涉及数万个有状态进程——GPU 线程、CPU 进程、NCCL 通信、存储客户端。框架的复杂度已经远超任何个人的理解范围。

系统越来越像一个"可控的黑箱"——我们理解它的配置接口，相信它能稳定工作，但已没有人能从代码层面完整理解整个训练栈的每一层。当你在万卡集群上训练万亿参数模型时，你只能测试出最稳定的配置组合然后信仰坚持，而不是理解每一行代码在做什么。

这可能是 AI 基础设施发展中最被低估的隐性成本：**我们正在用增长系统的不可理解性来换取算力的可扩展性。**

六、实战：面试题、场景题、应用题

面试题

Q1: ZeRO-3 和 TP 的区别是什么？各自解决什么问题？

考察对两种核心并行策略本质差异的理解，而不是复述概念。

核心区别：ZeRO-3 解决的是**显存冗余**问题，TP 解决的是**算力不足**问题。

ZeRO-3 的出发点是"数据并行中每张 GPU 都存了完整的参数/梯度/optimizer states，这是浪费的"。ZeRO-3 把这些数据分散到各 GPU，每张卡只持有一部分，需要时通过 all-gather 收集。ZeRO-3 不会加速计算——实际上因为通信开销，它会略微降低单步的计算速度。但它让无法放进单卡的模型可以训练。

TP 的出发点是"单个 Transformer 层的计算量太大，一张 GPU 算不完"。它把层的参数切开，多张 GPU 并行计算同一层的不同部分。TP 需要高频通信（每层 forward 两次 all-reduce），因此依赖 NVLink。TP 的实际价值是保持单步的低延迟——在 TP 下，算一个 Transformer 层的延迟可以比单卡低。

选型指南：

- 如果你的瓶颈是显存（模型太大放不下）→ ZeRO-3
- 如果你的瓶颈是算力（单卡计算太慢）→ TP
- 大模型训练中两者通常互补使用：TP 做模型内并行，ZeRO 做数据并行

Q2: 训练时 Loss 突然跳到 NaN，你从哪几个方向排查?

考察实际训练经验和系统排查能力，不看书面的知识。

排查层次（从最可能到最不可能）：

1. **数值溢出（最常见的原因）**：FP16 下某层的梯度太大，超出了 FP16 的表示范围（6.5e4）。检查：有没有在 loss scaling? loss scale 是多少? 如果是 FP8 训练，检查 block-wise scale 是否溢出。解决方案：提高 loss scale 的初始值，或者在 normalize 前增加 gradient clipping。
1. **学习率过大**：LR 太高，模型更新步长太大，跳过了最优区域。检查：当前学习率是多少? warmup 阶段有 loss spike 吗? 解决方案：降低 LR 或调整 warmup 步数。
1. **数据异常**：batch 中混入了损坏的数据（NaN label、inf features、过长的序列导致 attention softmax 溢出）。检查：复现时用确定的 seed + 固定的 batch 是否还 NaN? 如果在固定 batch 上复现，找到那个 batch 检查数据质量。
1. **模型结构问题**：LayerNorm 的 epsilon 太小（导致除零），或者某个激活函数在极端输入下的行为异常。检查：NaN 出现在特定的 layer 吗? 可以插入 `torch.isnan` 检查点逐层定位。
1. **通信错误**：all-reduce 或 all-gather 中数据损坏。在 ZeRO-3 中比较常见——参数收集后某个切片的 buffer 没有被正确更新。检查：单卡（DP=1）下是否还 NaN? 如果单卡正常但多卡 NaN，基本确定是通信问题。
1. **MoE 特有原因**：某个 expert 的参数突然变大，导致路由机制崩溃。检查：load balancing loss 的值是否异常? 是否有 expert "死掉"（所有 token 都不再路由到它）?

最实用的排查工具：炼丹师最应该学会的是"二分法"——先看是数据问题还是模型问题（换随机数据），再看是参数更新问题还是前向传播问题（用前一步的 checkpoint 做前向测试），最后逐层定位 NaN 源头。

Q3: Pipeline Parallelism 的 bubble 问题怎么解决?

考察对 PP 的理解深度——知道 PP 有 bubble 不够，还得知道怎么应对。

Bubble 的来源：流水线进入和退出阶段时，部分 GPU 处于空闲。PP=4 时有 ~25% 的 bubble，PP=16 时 ~40%。PP 越深，bubble 越严重。

解决方案（从简单到复杂）：

1. **Micro-batching（GPipe 方案）**：把每个 micro-batch 拆成更小的 micro-batch，让流水线更"满"。PP=4 + 16 个 micro-batch，气泡率 ~3%。代价是显存消耗增加（每多一个 micro-batch 就要多存一份中间激活）。这是最常用的方案。
1. **1F1B 调度**：一个 micro-batch 完成 forward 后立即开始退火（backward），不让 forward 阶段堆积。在保证跟 GPipe 相同 bubble 率的同时，显存占用降低一半。
1. **Interleaved Pipeline（交错流水线）**：每个 GPU 跑多个 stage（比如 PP=4 的流水线上，每个 GPU 跑两个 stage），让流水线上有更多并行执行的计算。气泡从 ~25% 降到 ~10%，但通信量翻倍。
1. **减少 PP 深度，增加 TP 深度**：DeepSeek-V3 的做法——PP 深度降到 2，主要用 TP + DP + EP。PP=2 的气泡只有 ~5%。当 NVLink 带宽足够时，这是一个更高效的选择。这个方案的工程前提是 NVLink 的拓扑能够支撑大 TP 深度。
1. **异步 PP（2025 年新兴研究）**：让 PP 的通信和计算完全重叠——类似 DualPipe 的思路但应用到 PP 场景。目前还没有主流框架原生支持。

Q4: 你要训一个 70B Dense 模型，4 节点 × 8 GPU，怎么选择并行策略?

考察实际决策能力——在给定硬件约束下的合理选择。

硬件：4 节点，每节点 8 GPU，共 32 GPU。每节点内 NVLink（900 GB/s），跨节点 100 Gbps Ethernet（~12.5 GB/s）。假设 GPU 是 H100 80GB。

模型：70B Dense Transformer（80 层，d=8192，FFN 中间维度 28672，GQA 8 KV heads）。

方案：

TP=8 + PP=1（节点内），DP=4（跨节点）。每个节点内部 8 张 GPU 做 TP（充分利用 NVLink），4 个节点通过 DP（Ethernet）同步梯度。

显存计算：70B FP16 = 140GB。TP=8 下每卡 140/8 ≈ 17.5GB 参数。加上梯度 ~17.5GB，optimizer states（Adam FP32）~70GB——等等，这已经超了 H100 80GB 的容量了！

所以需要使用 **ZeRO-1 或 ZeRO-2**。DP=4 下，ZeRO-1 把 optimizer states 切到 4 张 GPU，每卡 ~17.5GB。DP + ZeRO-1 的显存分布：参数 17.5 + 梯度 17.5 + optimizer states 17.5 = 52.5GB。加上中间激活和大容量 KV cache（~10-15GB），总计约 65-70GB。可以跑，但余量不大。

更好的方案：TP=8 + ZeRO-1 DP=4，加上可选的 activation checkpointing（用计算换显存，省掉 ~30% 的激活缓存）。global batch size = 32 sequences × 4096 tokens。

不推荐的方案：PP。PP 在 32 GPU 上深度太小（PP=2）浪费潜力，深度大（PP=8）气泡太多。4 节点互联的 Ethernet 带宽也不适合 PP 的高频 stage 间传输。

Q5: DeepSeek-V3 的 DualPipe 解决的是什么问题?

考察对训练系统前沿的理解——不是所有人都需要知道 DualPipe 的细节，但应该有框架性的理解。

问题：MoE 训练中，all-to-all 通信是主要瓶颈。每个 Transformer 层的 MoE block 有两次大通信——dispatch token 到对应 expert（all-to-all）和 combine expert 的输出（all-to-all）。在 H800 上，一次 all-to-all 约 5-10ms，而 MoE block 的计算大约 10-15ms——通信占了 30-50% 的时间。

DualPipe 的方案：不让通信和计算串行执行，而是让它们重叠。具体做法是把计算和通信编排成双流水线：在一个 micro-batch 的 MoE block 计算中，同时启动下一个 micro-batch 的 all-to-all 通信。这跟在 CPU 中把指令级流水线的执行阶段和访存阶段重叠是同一个思路。

效果：MoE 训练的通信延迟几乎完全隐藏。"free lunch"——不增加硬件，不增加计算量，只改调度顺序，等效于多了 25-30% 的算力。

更深层的洞察：DualPipe 的成功说明 MoE 训练中"显式的通信等待"是最大的浪费。所有分布式训练框架都应该检查自己的调度模式，找到"GPU 在空等通信"的窗口期，然后用计算把它填满。

场景题

场景 1：并行策略选择

> 一个初创公司想在 32 张 A100 上训 13B 模型，FSDP vs DeepSpeed 怎么选?

约束：初创团队，没有专职 infra 人员，模型 13B（不算太大），32 张 A100 可能是云上的（无法保证拓扑）。

推荐：FSDP。

理由：

1. **零配置启动**。torchrun + FSDP wrapper 一行代码能跑起来。DeepSpeed 需要写 ds_config.json、指定 zero stage、配置通信参数——调试成本高。
1. **13B 不需要 3D 并行**。13B FP16 约 26GB，单卡都能放下。32 张 A100 纯 FSDP（相当于 ZeRO-3）每卡只需 26/32 ≈ 0.8GB 参数——显存极度充裕。
1. **云环境拓扑不确定**。DeepSpeed + Megatron 的 TP 依赖 NVLink，如果在云上租到的是跨节点 GPU（没有 NVLink 互联），TP 效率极低。FSDP 不依赖 NVLink。
1. **未来可扩展**。模型成长到 70B 级别时，FSDP 的 hybrid shard 模式可以无缝适配——节点内不切分（full replication），节点间切分（ZeRO-3）。不需要换框架。

不推荐 DeepSpeed 的理由：DeepSpeed 的优势在大模型（100B+）场景下，ZeRO-3 的显存节省和 MoE 支持才有决定性差异。13B 规模，DeepSpeed 的复杂度 > 它的价值。

场景 2：显存估算

> LLaMA 70B（80 层，d=8192，FFN=28672），FP16 训练。Adam optimizer + ZeRO-3。一张 H100 80GB 能放几层?

前提：ZeRO-3 下参数、梯度、optimizer states 全部切分。假设 DP=8（即 8 张卡）。

第一步：单层参数量。

Attention 层：Q、K、V、O 四个投影矩阵。d=8192，GQA 8 KV heads（即 Q heads 64，KV heads 8）。d_head=128。

- Q: 8192 × (64×128) = 8192 × 8192 = 67,108,864 = 67.1M
- K: 8192 × (8×128) = 8192 × 1024 = 8,388,608 = 8.4M
- V: 同 K, 8.4M
- O: (64×128) × 8192 = 同 Q, 67.1M

Attention 总参数量: 67.1 + 8.4 + 8.4 + 67.1 = 151M

FFN 层：三个矩阵（gate、up、down），类似 SwiGLU。d=8192, FFN=28672。

- gate: 8192 × 28672 = 234.9M
- up: 8192 × 28672 = 234.9M
- down: 28672 × 8192 = 234.9M

FFN 总参数量: 704.7M

单层总参数量：151M + 704.7M ≈ 856M = 0.856B

第二步：单层显存消耗（FP16，DP=8, ZeRO-3）。

FP16 参数：0.856B × 2 bytes = 1.71GB。DP=8 + ZeRO-3 下，每卡只需 1.71/8 = **0.214GB**。

梯度：同参数，0.214GB。

Optimizer states（Adam FP32）：0.856B × 4 bytes × 2（momentum + variance）= 6.85GB。ZeRO-3 下每卡 6.85/8 = **0.856GB**。

每层合计（参数 + 梯度 + optimizer）：0.214 + 0.214 + 0.856 = **1.284GB**。

第三步：其他开销。

中间激活（activation）：这是大头。每层前向传播需要缓存激活用于反向传播计算。粗略估算每层约 2-4GB（取决于 batch size 和序列长度）。假设 batch size=1, seq_len=4096，activation per layer ≈ 3GB。

KV cache（训练时也需要缓存 K、V 用于 backward 时 re-compute attention）：~0.5GB per layer（跟推理不同，训练时 K、V 不需要完整缓存，但需要缓存更多中间状态）。

第四步：上限计算。

H100 80GB 实际可用显存约 75GB（预留 CUDA context、框架缓存）。假设 activation checkpointing（用额外前向计算换显存节省），activation 占用降到约 50%。

每层显存 = 1.284（模型数据）+ 3×0.5（activation checkpointing）= ~2.8GB。

75GB / 2.8 ≈ **26 层**。

结论：单张 H100 80GB，ZeRO-3 + DP=8 + activation checkpointing，约能放下约 26 层。80 层 LLaMA 70B 需要 80/26 ≈ 3-4 张 GPU 来放模型，加上跨层通信和流水线开销，实际训练至少需要 4-8 张 GPU。

注意：这个计算极度依赖 activation checkpointing 的配置。如果不用 checkpointing，activation 直接翻倍，单卡只能放 15 层左右。

场景 3：MoE 训练设计

> 要训一个 MoE 模型（总参数 ~200B，激活 ~20B，16 experts），怎么设计并行策略？怎么处理 all-to-all 通信？

第一步：参数分布。

总参数 200B，FP16 下 400GB。16 experts，每个 expert 约 12.5B。Dense shared layers（embedding、output head）约 10B（非 expert 参数）。

第二步：硬件假设。假设有 64 张 H100 80GB，分布式在 8 个节点上（每节点 8 卡，NVLink 互联）。

第三步：并行策略。

- **TP=8**（节点内）：每个节点 8 卡做张量并行。Dense layers 和 expert layers 都可以用 TP——expert 的参数在 TP 范围内是完整的（每张卡存一个 expert 的切片）。
- **EP=8**（跨节点）：16 experts 分布到 8 个节点，每个节点负责 2 个 expert。token 根据路由结果，通过 all-to-all 发送到目标节点对应的 expert。
- **DP=2**（跨节点）：2 路数据并行（实际上因为 EP=8 已经用满了 8 个节点，DP 在 EP 基础上再做一次扩展——如果 GPU 数量翻倍到 128，DP=2，EP 不变）。

并行组合：TP=8 × EP=8 × DP=2（实际总 GPU 数 = 8 × 8 × 2 = 128，这里跟我们假设的 64 GPU 有出入——实践中会根据 GPU 数量动态调整）。

针对 64 GPU：TP=8（节点内）+ EP=8（跨节点）+ DP=1（64 GPU 刚够放 200B 参数，没有多余的卡做纯 DP）。实际上需要更多 GPU——200B FP16 模型 400GB，64 × 80GB = 5120GB，减去 CUDA context 和 activation，刚好够。

第四步：All-to-all 通信优化。

16 experts 分布在 8 个节点上，每个 token 取 top-2 experts。All-to-all 通信分两步：

1. Dispatch：每张卡把 token 发给目标节点的目标 expert（token relocation）
2. Combine：expert 计算完成后把结果发回原卡

优化方案：

- **通信-计算重叠**：在 MoE block 前先做 all-to-all dispatch，在 MoE block 计算的同时做下一个 all-to-all 的 dispatch——类似 DualPipe 的微调版。
- **分层 all-to-all**：节点内的 all-to-all（NVLink，极快）和跨节点的 all-to-all（Ethernet，较慢）分开处理。如果 token 的 expert 在同一个节点内（NVLink），不走 Ethernet 通信。
- **Token coalescing**：多个 token 拼成一个大的 message 发送，减少 message 数量，提高 Ethernet 的带宽利用率。
- **负载均衡**：使用 DeepSeek 的 aux-loss-free 动态 bias 调整，确保 expert 的负载尽量均匀——因为负载不均意味着 all-to-all 通信量的分布不均，最忙的卡会成为瓶颈。

第五步：预估 MFU。

假设所有优化到位，MFU 估计在 50-65% 之间（低于 DeepSeek-V3 的 70%，因为我们的 MoE 规模更大，all-to-all 占比更高）。

应用题

应用 1：设计一个训练调度方案

> 你手里有 128 张 H100，要训一个 DeepSeek-V3 规模的模型（671B MoE）。从框架选择、并行策略、通信优化、容错策略给出完整方案。

背景：671B MoE 模型，37B 激活参数，256 experts（DeepSeek-V3 的实际配置是 256 experts，top-8 routing）。FP8 训练。目标 MFU >60%。

硬件：128 张 H100 80GB，分布在 16 个节点（每节点 8 卡）。节点内 NVLink 900 GB/s。跨节点 InfiniBand 400 Gbps（~50 GB/s）。

框架选择：DeepSpeed + Megatron-LM 组合。FSDP 在这个规模下不支持。DeepSpeed 负责 ZeRO 和 MoE 支持。Megatron 负责 TP 和 PP。FP8 训练需要使用 DeepSpeed 的 FP8 训练支持（或者自己实现——DeepSeek 的 FP8 训练方案是自研的）。

并行策略：

- **TP=8**：节点内 8 卡做张量并行。每张卡持有一个 Transformer 层 1/8 的参数。NVLink 足够支撑 TP 的高频通信。
- **PP=2**：模型分成 2 个 stage。每个 stage 由 8 卡 TP 组完成。跨 stage 通信通过 InfiniBand 的 point-to-point。
- **EP=8**：256 experts 分布到 16 × 8 = 128 张卡上。但每张卡不只是负责 expert——每张卡同时还负责 TP 的切分。所以实际上 expert 的分布是：每个节点（8 卡 TP 组）负责 256/8 = 32 个 experts（每个卡负责 32/8 = 4 个 experts 的切片）。
- **DP**：在 EP 和 TP 已经用满所有 GPU 的情况下，DP 不可用或者可以跟 EP 混合（把一组 EP 看成 DP 的一个 replica）。

实际配置（精确到每张卡的角色）：

每张 H100 需要同时：

1. 持有 1/8 的 Dense layer 参数
2. 持有 4 个 experts 的 1/8 参数
3. 参与 TP 组内的 all-reduce 通信（每层 2 次）
4. 参与 EP 的 all-to-all 通信（每 MoE block 2 次）
5. 参与 PP 的 point-to-point 通信（每 model stage 1 次）

通信优化：

1. **DualPipe 方案**：采用 DeepSeek 的双流水线调度，让 MoE 的 all-to-all 通信与 backward 计算重叠。
2. **分层通信**：所有 TP 通信走 NVLink（节点内），PP 和 EP 通信走 InfiniBand。确保跨节点通信不要用 NVLink——也不会，InfiniBand 不支持 NCCL 的 NVLink 通信。

3. **FP8 all-reduce**：梯度 all-reduce 用 FP8 精度，减少 50% 的通信量。DeepSeek 的论文中证明了 FP8 梯度 all-reduce 对收敛的影响可忽略。
4. **Async gradient sync**：DP（在跨 EP replica 间）的梯度同步使用异步模式，与 PP 的 backward 计算重叠。

容错策略：

在 128 张 H100 上训练 671B 模型，预期出现故障的频率：约每 3-5 天一次。

1. **Checkpoint 频率**：每 1 小时一个完整 checkpoint，每 15 分钟一个 partial checkpoint（只存 optimizer states 和 LR 调度器状态）。Checkpoint 大小：FP8 参数 671GB + optimizer states ~2TB（FP32 momentum + variance）= ~2.7TB。写速度取决于存储后端的 IO。如果存储 IO 是 10 GB/s，单次 checkpoint 约 270 秒（4.5 分钟）。在 HF 高频 checkpoint 不现实，所以每 1 小时和每 15 分钟的折中方案是合理的。

1. **弹性训练（Elastic Training）**：支持热替换故障节点。128 卡中坏掉 1 张，不停训整体，只重新分配故障节点上 expert 和 TP 组到备用节点。DeepSpeed 的 elastic training 功能在 2025 年已支持类似场景——备用节点自动加入 NCCL group，梯度同步从 128 卡调到 127 卡。

1. **Loss spike 自动检测**：在训练循环中加入 NaN/inf 检测器。如果 loss 异常，自动回退到上一个 checkpoint + 降低学习率再训练。

1. **通信超时**：每步通信设置 120 秒超时。超时后自动跳过该步（skipping step），不中断训练。这比"所有卡等一张卡"的方案更激进，但 DeepSeek 在实践中证明了跳过个别 step 对最终收敛无影响。

MFU 预估：

TP=8 的通信开销：~5%（因为 NVLink 带宽充足，all-reduce 几乎不占计算时间）。

PP=2 的 bubble 开销：~5%。

EP all-to-all 开销（经过 DualPipe 优化后）：~10%。

FP8 计算效率：H100 FP8 tensor core 利用率约 80%（相比于理论峰值）。

MFU 预估：1 × 0.95（TP comm）× 0.95（PP bubble）× 0.90（EP comm）× 0.80（FP8 efficiency）= **~65%**。

这个数字不是最好（DeepSeek-V3 做到了 ~70%），但已经很好了。剩余的优化空间主要在于进一步压缩 EP 的 all-to-all 通信开销（比如更好的 token 合并策略，或者探索异步 all-to-all 的可行性）。

七、信息来源

核心论文与文章

论文/文章	来源	访问时间
Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models" (2020)	arxiv.org/abs/1910.02054	2026-04-30
Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism" (2020)	arxiv.org/abs/1909.08053	2026-04-30
Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM" (2021)	arxiv.org/abs/2104.04473	2026-04-30
Brown et al., "Language Models are Few-Shot Learners" (GPT-3, 2020)	arxiv.org/abs/2005.14165	2026-04-30
Chowdhery et al., "PaLM: Scaling Language Modeling with Pathways" (2022)	arxiv.org/abs/2204.02311	2026-04-30
Touvron et al., "LLaMA: Open and Efficient Foundation Language Models" (2023)	arxiv.org/abs/2302.13971	2026-04-30
DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model" (2024)	arxiv.org/abs/2405.04434	2026-04-30
DeepSeek-AI, "DeepSeek-V3 Technical Report" (2024)	arxiv.org/abs/2412.19437	2026-04-30
DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via RL" (2025)	arxiv.org/abs/2501.12948	2026-04-30
Micikevicius et al., "Mixed Precision Training" (2018)	arxiv.org/abs/1710.03740	2026-04-30
Micikevicius et al., "FP8 Formats for Deep Learning" (2022)	arxiv.org/abs/2209.05433	2026-04-30
Lepikhin et al., "GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding" (2021)	arxiv.org/abs/2006.16668	2026-04-30
Xu et al., "1-bit Adam: Communication Efficient Large-Scale Training with Adam's Convergence Speed" (2021)	arxiv.org/abs/2102.02888	2026-04-30
Fedus et al., "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity" (2022)	arxiv.org/abs/2101.03961	2026-04-30
Zhao et al., "Exascale Deep Learning for Scientific Inverse Problems" (2023)	arxiv.org/abs/2309.05787	2026-04-30
Ren et al., "ZeRO-Offload: Democratizing Billion-Scale Model Training" (2021)	arxiv.org/abs/2101.06840	2026-04-30

Rajbhandari et al., "ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning" (2021)	arxiv.org/abs/2104.07857	2026-04-30
Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness" (2022)	arxiv.org/abs/2205.14135	2026-04-30
Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning" (2023)	arxiv.org/abs/2307.08691	2026-04-30
Shah et al., "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision" (2024)	arxiv.org/abs/2407.08608	2026-04-30
Jacobs et al., "Colossal-AI: A Unified Deep Learning System For Large-Scale Parallel Training" (2021)	arxiv.org/abs/2110.14883	2026-04-30
Zheng et al., "Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning" (2022)	arxiv.org/abs/2201.12023	2026-04-30
DeepSpeed Ulysses (2023)	arxiv.org/abs/2309.14509	2026-04-30

产品与技术来源

来源	URL	访问时间
DeepSpeed 官方文档	https://www.deepspeed.ai/	2026-04-30
Megatron-LM 仓库	https://github.com/NVIDIA/Megatron-LM	2026-04-30
PyTorch FSDP 文档	https://pytorch.org/docs/stable/fsdp.html	2026-04-30
JAX 官方文档	https://jax.readthedocs.io/	2026-04-30
DeepSpeed MoE 文档	https://www.deepspeed.ai/tutorials/moe/	2026-04-30
"The Infrastructure Behind AI: Large-Scale Training" — SemiAnalysis (2024)	SemiAnalysis	2026-04-30
"DeepSeek-V3: Everything You Need to Know" — Latent Space (2025)	Latent Space	2026-04-30
"Understanding DeepSeek-V3's FP8 Training" — interconnects.ai (2025)	interconnects.ai	2026-04-30
"Scaling Laws vs. System Efficiency" — Gradient Flow (2025)	Gradient Flow	2026-04-30
"Why DeepSeek-V3 Training Cost So Little" — Evan Miller (2025)	https://evanmiller.org/deepseek-v3-training-cost.html	2026-04-30
"The Cost of Training LLMs" — Stanford CRFM Blog (2025)	Stanford CRFM	2026-04-30
"FSDP vs DeepSpeed" — PyTorch 官方博客 (2024)	https://pytorch.org/blog/fsdp-vs-deepspeed/	2026-04-30
"Distributed Training at Scale" — Microsoft Research Blog (2024)	Microsoft Research	2026-04-30
"The Return of Dense Models?" — Yuchen Jin, 知乎 (2025)	知乎	2026-04-30
ColossalAI 开源仓库	https://github.com/hpcaitech/ColossalAI	2026-04-30
DeepSpeed 开源 + ZeRO 发布	Microsoft Research Blog (2020-02)	2026-04-30

Megatron-DeepSpeed 合作	Microsoft Research Blog (2021-05)	2026-04-30
Megatron Core 发布	NVIDIA Developer Blog (2024-07)	2026-04-30
Llama 3 训练基础设施	Meta AI Blog (2024-07)	2026-04-30

> **方法论说明**：本报告使用横纵分析法（Horizontal-Vertical Analysis），由数字生命卡兹克提出。纵向追时间深度（从 2018 年混合精度训练到 2025 年 DeepSeek-V3 的完整发展历程），横向追同期广度（四大训练框架、并行策略全景、MoE vs Dense 全面对比），最终在交叉处产出新的判断：训练效率提升的主要驱动力正在从硬件转向算法和系统，分布式训练的本质是不断在约束之间转移矛盾而非消除矛盾。

> 写作沿用了 Transformer 推理报告和 MoE 报告的叙事风格——以具体事件和决策为锚点，在技术细节与行业叙事之间寻找平衡，避免咨询报告式的空洞概括。

本文由"Transformer 训练"和"大模型训练框架与基础设施"两篇报告合并而成。方法论：横纵分析法——纵向沿时间轴追溯技术演进，横向对比当下竞争格局。